What the Final Patch Hides: Event-Level Regression Measurement for Coding-Agent Harnesses

Anonymous Author(s)
Affiliation
Address
email

Abstract

When a coding agent is graded, only its final patch is examined. We asked what happens in between: does the agent break the repository's existing tests while it works, and does the final patch show it? We built an instrument that snapshots the working tree after every edit and re-runs the repository's previously-passing tests at every snapshot inside the benchmark's own container. In 47 runs of one frontier stack under a rollback policy on 40 SWE-bench Verified tasks (a 37-run sweep plus a 10-task calibration), the timeline recorded 184 regression events; the final patches showed one. On the same 40 tasks a second frontier stack produced no events in 40 runs, and the first produced 140 in its 37 sweep runs; on a 40-task SWE-bench Live slice (39 graded), a rolling set built from live repositories, both stacks broke tests, the timeline again recorded events the final patches did not show (156 under rollback, none visible), and resolve rates fell to near zero for every arm. Rolling back to the last verified checkpoint keeps the final tree clean but costs solved tasks, because the check that triggers the rollback rejects correct patches. On Verified, gating each step on the repository's own tests removes that cost: 65% solved with no contaminated final tree, and 70% when the gate reads only half of those tests and the grader checks the other half. We will release the instrument and its validation protocol; Appendix A catalogues 28 measurement defects met while building it, 23 of which produced plausible numbers rather than errors.

1 Introduction

Benchmarks for coding agents grade the final patch: SWE-bench [1] and its Verified subset [2] apply the submitted patch, run the repository's tests, and report whether the target tests pass and the previously passing tests still pass; work on agent-caused regressions counts the previously-passing tests the submitted patch breaks [8].

That convention cannot see what happens while the agent works. An agent that breaks an existing test at step three and repairs it at step five submits a clean patch; any harness with a recovery mechanism produces that pattern by design. Two basic questions therefore have no answer: how often do agents break existing behaviour during a run, and how much of it survives to the final patch?

We answer both by measuring the timeline instead of the endpoint. Every mutating tool call becomes a commit on a git reference the agent cannot see; after the run, the repository's previously-passing tests are re-run at every commit inside the benchmark's pinned container. This is the observation primitive an agent operating system needs before it can build a recovery primitive. SWE-bench Verified is deprecated as a capability metric [46]; we use it as a measurement substrate, and Section 5.7 reports the same measurement on SWE-bench Live. Our contributions are:

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1. **An instrument for event-level regression measurement.** Every mutating tool call is recorded as a commit on a git reference the agent cannot see; after the run, the instance's previously-passing tests are replayed at every recorded state inside the benchmark's pinned container, and detection is attributed by coverage. Validity is established by controls, a liveness check, a golden check, and a re-scoring control (Section 3).

2. **Measurements on SWE-bench Verified.** Across the GPT-5.6 rollback runs that produced any regression, the timeline recorded 184 events and the final state exposed one (Section 5.2). On Verified, event frequency depended on the model stack: the same 40-instance slice (37 attempted under GPT-5.6) and harness gave 0 events under one frontier stack and 140 under another (Section 5.3). On SWE-bench Live the undercount replicated and the stack difference was not detected (Section 5.7).

3. **A pre-declared, exploratory comparison of recovery policies, and a fix.** Rollback to a verified checkpoint left fewer contaminated final trees than no recovery ($p = 0.375$ at first unblinding, $p = 0.022$ after a post-unblinding grading correction, Section 5.4) but resolved fewer tasks; we trace the loss to the monitor's false rejections, and, on Verified, a harness-enforced regression gate removes it: 65% resolve with no contaminated tree, 70% when graded on tests the gate never reads (Sections 5.4 to 5.6).

4. **A catalogue of 28 measurement defects** found while building the instrument, classified by mechanism (Appendix A); 23 produced a plausible number rather than an error.

All measurements are exploratory, made on development slices excluded from any future confirmatory study; no claim is made beyond those slices.

2 Background and related work

Benchmarks and their validity. SWE-bench [1] grades a patch by tests that must go from failing to passing and tests that must keep passing; Verified [2] is the human-screened 500-instance subset. Concerns about the static subset have accumulated: training-data contamination, memorisation, and leaked solutions [6]; insufficient tests, which UTBoost [7] augmented to expose 345 mislabelled patches affecting 24.4% of Verified leaderboard entries; and rolling [3, 5], private [4], or long-horizon [23] alternatives in response. Wang et al. [37] find resolve rates on Verified overstated by 6.2 points once plausible-but-wrong patches are inspected. TDAD [8] measured regressions in submitted patches on Verified (6.1% of runs under a vanilla open-weight agent) and reduced them with a pre-change impact-analysis map that tells the agent which tests to verify. Process-level benchmarks have begun to score intermediate states from logs: RigorBench [35] on 30 curated tasks, SWE-CI [36] across CI iterations, AgentLens [38] naming regression cycles in 10.7% of passing OpenHands runs. We measure the same quantity at every mutating call on standard instances, by executed replay.

Agent scaffolds and recovery. SWE-agent [9], OpenHands [10], Agentless [11], and mini-swe-agent [12] span the scaffold design space, built on CodeAct [13] and ReAct [14]. Within-episode recovery has been studied as self-reflection [15, 16], progress-gated recovery [17], checkpoint repair [18], and aligned checkpoints [41]; provenance-based recovery is surveyed in [19]. Closest to us, Kim et al. [39] match the logged intermediate edits of 16,758 trajectories against the gold diff, re-execute the five exact matches, find agents that reach a gold-identical patch and then destroy it, and recover those cases with an edit-commit checkpoint; Gao et al. [40] bind verifier evidence to exact code states and preserve verified checkpoints on function-level repairs. Neither counts regressions or compares recovery policies. Running the repository's regression tests during a run is deployed practice: TestPrune [43] offers the agent a minimised suite (8 to 13% relative resolve gain across three scaffolds) and Trae Agent [44] filters candidates with regression tests. Operating-system framings [20] and the branch-context primitive [42] supply scheduling and fork, commit, and abort, but do not observe the tree between actions.

Automated program repair. The regression problem is the plausible-versus-correct patch problem of program repair [26, 28, 29]; regression tests are the main defence against overfitting patches [27]. Our results concern regressions the agent's own process repairs before the patch is tested; trajectory-level evaluation [21, 22] argues that resolve rate alone is uninformative. Public leaderboard trajectories record actions but not the tree at each step; the instrument commits the tree.

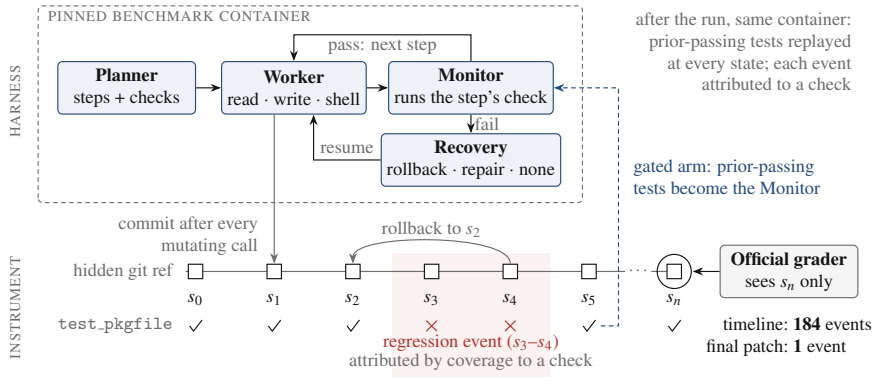

Figure 1: The harness and the instrument. The planner, worker, monitor, and recovery policy run inside the benchmark's pinned container; every mutating tool call commits the tree to a hidden git reference, and after the run the instance's previously-passing tests are replayed at every state. The test row shows one such test regressing at s_3–s_4 and repaired by a rollback to s_2, which the official grader, reading only s_n, never sees. In the gated arm (dashed), the timeline's tests serve as the monitor.

3 Instrument

Figure 1 shows the harness and the instrument. **Observational timeline.** After every mutating tool call the working tree is committed to a git reference outside the agent's view (a private index keeps the agent's own `git status` unchanged); rollbacks and the end of the run are observations too, and an archived run can be re-measured without re-running the agent.

Exhaustive replay. For each observation, the instance's passing-test set is run inside the pinned container against that observation's tree; a regression event is a test that passes at one observation and fails at a later one. Every observation is replayed rather than bisected: a recovery policy makes verdicts non-monotone. Replay is scoped to the files holding the held-out tests, which keeps it affordable on Verified: 4 to 8 seconds per observation (pytest, django), about 26 CPU-minutes per 40-instance sweep; Live's larger oracles cost tens of minutes per instance. Infrastructure failures during replay are recorded as missing observations rather than as test failures, and tests that already fail in the unmodified image (baseline-dead) are typed as missing observations for event counting.

Attribution. A detected regression is classified three ways: *attributed* if a failing harness check and the broken test both exercise a file the agent changed at that observation (per-instance coverage map at the base commit), *co-occurring* if some check failed while the regression was open without such a link, and *unknown* if the map cannot say.

Execution environment. The agent's tools and the harness's checks execute inside the same pinned container as the replay, with file changes synchronised to the host tree the timeline records (Appendix A.2).

Validation. Five checks run before any paid experiment: a negative control, a positive control with injected regressions including recovered ones, a flake screen, an unknown-rate ceiling, and a baseline liveness check. A golden check drives the gold patch through the agent's real tool path (the grader must return resolved, a null run unresolved); it passed on three repositories per benchmark (requests, django, matplotlib; cfn-lint, pvlib, reflex). A re-scoring control re-measures an archived timeline under a changed instrument with no model calls; on the two archived timelines re-scored so far it reproduced the archived regression counts exactly.

4 Experimental setup

Benchmark and slice. SWE-bench Verified, 500 instances. It is no longer a capability benchmark: its maintainers' audit found saturation, training-data contamination, and tests that reject correct patches [46]. We use it as a substrate, not a leaderboard: its pinned images and previously-passing test sets make the measurement possible. A 40-instance development slice (16 from django), stratified and

fixed before any measurement, was used throughout and is excluded from any future confirmatory study; the GPT-5.6 stack also ran on its first 10 instances as a calibration, and plain rollback ran twice. SWE-bench Live [3] is the second substrate: a rolling set built monthly from live repositories (this slice: pull requests from October and November 2024), graded by its own harness, whose rules we mirror (an xfail counts as a failure; a test absent from the log is not counted as a failure). Its slice was fixed before any outcome (pytest-parsed instances with 27 to 3,000 previously-passing tests, at most four per repository, earliest first; the first 40 of 48 candidates whose image passed a model-free environment check). Live oracles on this slice are about twenty times the Verified slice's (median 1,189 against 58 previously-passing tests); no coverage maps are built for Live, so attribution there is unknown by construction.

Harness. A planner decomposes the task into steps with verification commands; a worker executes each step with three tools (read, write, shell); a monitor runs the verification, and on failure the recovery policy acts: **rollback** (reset to the last verified checkpoint), **repair-in-place** (retry from the failed tree), or **no recovery** (keep the failed step's tree). Runs have a $4 work-cost cap; exhaustion scores as failure. One instance run under one policy is a cell; each (model stack, policy, substrate) triple is an arm.

Models. Two frontier stacks under identical harness, policy, instances, and caps, named by their API model strings: `claude-opus-4-7` (planner) with `claude-sonnet-4-6` (worker), and `gpt-5.6-sol` (planner) with `gpt-5.6-terra` (worker), recorded in every run manifest.

Outcomes. Resolve is the official grader's verdict on the final patch. Events are reported at three levels because a few runs produce most events: incidents (observations at which at least one test broke; their count per run, incident exposure, is the co-primary endpoint), declared events (the pre-declared unit: one per test function and onset, parametrised variants collapsed), and bearing runs (runs with at least one event). Final-state contamination is the number of previously-passing tests failing in the graded final patch, net of baseline-dead tests. A final tree that kills the suite is graded as failing every test (the official rule) but is a missing observation for event counting, so a contaminated cell need not be a bearing run (five no-recovery and two repair-in-place trees on Verified); a graded test absent from the log is scored failed, as upstream does.

Pre-declaration. The event unit, the proceed criteria, and the recovery-policy contrast were declared before the relevant data existed; the contrast's analysis (exact paired sign tests, final-state contamination primary, incident exposure co-primary) was committed before either comparison arm finished; one amendment, making the proceed criteria conditional on the model stack, preceded unblinding, and one grading correction followed it (Section 5.4, Appendix B).

5 Results

5.1 Resolve and cost

Of the 40 GPT-5.6 cells, a circuit breaker stopped three before they ran and a file-synchronisation defect lost two, so 37 were attempted and 35 graded; rates are out of 40 unless stated. Under rollback, the Claude stack resolved 13 of 40 (32.5%, exact 95% CI 18.6% to 49.1%) for a total sweep cost of $34.24; the GPT-5.6 stack 13 of 40 (37.1% of graded) for $8.14. One instance cannot be resolved by any arm under the current official parser (Appendix A.3), so every rate has a ceiling of 39. Runs are short: the median run makes 5 mutating tool calls (IQR 4 to 8; 94% make 10 or fewer), ten times fewer than public-scaffold trajectories.

5.2 Under rollback, the final state hides almost all regression activity

Figure 2 shows every run with a regression event under GPT-5.6 rollback, pooling the 10 calibration runs and the 37 attempted sweep runs. The timeline recorded 184 declared events across eight runs; the final patch exposed one. Seven of the eight bearing runs ended clean; three storms (73, 48, and 41 events) supply 88% of the events, so the run-level statement is the more robust one. Of the calibration's 57 raw episodes, 39 were attributed to a failing check, 16 co-occurred with one, 2 were silent; 48 of the sweep's 162 had no co-occurring failure.

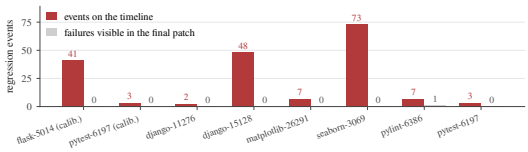

Figure 2: Every run that produced at least one regression event in the 10-instance calibration and the first 40-instance GPT-5.6 rollback sweep: events recorded on the timeline (red) against previously-passing test failures visible in the graded final patch (grey).

5.3 On Verified, regression frequency differs by model stack

On the same 40 instances, under the same harness and rollback policy, the Claude stack produced no regression events in 40 runs (227 observations, all replayed). The GPT-5.6 stack produced 140 declared events in 11 incidents across 6 of 37 attempted runs (bearing fraction 16.2%, exact CI 6.2% to 32.0%). A second plain-rollback sweep reproduced this (13 of 36 graded resolved, 116 events, 6 of 37 bearing, $6.87; paired contamination differed on one instance and incident exposure on five, sign tests p = 1.0). A one-sided Fisher exact test on the bearing fractions gives p = 0.010; this is a single pairwise test on six bearing runs (reassign three of them and p = 0.11), and the two stacks are called through different provider client code, so model and adapter are confounded. At equal resolve (Table 1), the stack with no regression events cost four times as much. The two sweep storms (121 of 140 events) and the calibration's flask storm are model edits, not harness artifacts.

The Claude zero is measured, not a detection failure: re-scoring the same stack's earlier canary run reproduces its three events.

The difference is not detected on SWE-bench Live (Section 5.7).

5.4 Recovery policy: rollback keeps the final tree clean

Table 1 summarises the pre-declared contrast. On the primary endpoint, final-state contamination paired by instance, no recovery was worse than rollback on 9 instances and better on 1 (exact sign test p = 0.022); repair-in-place was worse on 5, better on 1 (p = 0.22). Incident exposure, the co-primary, did not differ on the observations the replay could score (p = 0.45 and 1.0): regressions occurred at similar rates under every policy; the policy determined whether they persisted.

Disclosure. The first unblinding gave p = 0.375: seven cells (five no-recovery, two repair-in-place) whose final trees killed the suite had been dropped as ungradable, removing the most contaminated states from the endpoint, whereas official grading scores such a patch as failing every test; the grader was corrected and the seven archived trees re-graded without re-running any agent, giving p = 0.022, a rule change made after the data were seen (Appendix B).

5.5 Rollback loses resolution to the monitor's false rejections

Rollback resolved 13 of 40 against 24 for repair-in-place and 22 for no recovery (Table 1); paired by instance it lost nine discordant pairs and won one against repair-in-place (McNemar p = 0.022) and lost nine, won three against no recovery (p = 0.15). The loss is the monitor's: of the 18 rollback runs that failed, 15 failed at the first step, and 9 of those 18 instances were resolved by an arm that kept the rejected work: the planner-written check had rejected a patch the grader accepted.

5.6 On Verified, regression-gated rollback removes the tradeoff

A fourth arm, added after unblinding and therefore exploratory, replaces the monitor's planner-written check with the repository's previously-passing tests, run in the agent's container after every attempt; a step is rejected only if a test that passed at the start fails now. It resolved **26 of 40 instances (65.0%)**, above rollback's 13 and the ungated arms' 24 and 22, at the level frontier scaffolds report ungated [39, 44], with **no contaminated final tree** (Table 1). Against plain rollback on 35 shared instances it won 10 discordant pairs and lost 1 (McNemar p = 0.012, post hoc, unadjusted; incident exposure p = 0.73). This result is partly circular: the gate's tests are the benchmark's own previously-passing set (median ratio to the grading oracle 1.00), so a clean final tree follows whenever the gate evaluated the

Table 1: Every arm on both substrates: resolved of 40 (Live: its official rule; in parentheses, rot-aware: every target test passes and no failure lies outside the baseline-dead set), declared events, incidents, bearing runs over runs attempted, final trees failing a previously-passing test beyond the baseline-dead set, total spend. Claude Verified's 1 is a test absent from the grading log (failed by upstream's rule) with no timeline event; no recovery's 9 and repair-in-place's 5 include five and two suite-killed trees with no timeline event; Live counts exclude one instance whose suite exceeds the instrument's budget.

substrate	arm	resolved	events	incidents	bearing	contaminated	spend
Verified	GPT-5.6 rollback	13	140	11	6/37	1	$8.14
Verified	Claude rollback	13	0	0	0/40	1	$34.24
Verified	GPT-5.6 gated	26	70	7	5/40	0	$10.51
Verified	GPT-5.6 split-gated	28	14	4	3/40	0	$9.40
Verified	GPT-5.6 repair-in-place	24	84	7	7/40	5	$14.73
Verified	GPT-5.6 no recovery	22	19	3	3/40	9	$4.40
Live	GPT-5.6 rollback	0 (2)	156	9	6/39	0	$10.29
Live	Claude rollback	2 (3)	49	6	3/39	1	$49.80
Live	GPT-5.6 gated	1 (4)	280	6	3/39	2	$11.49
Live	GPT-5.6 no recovery	4 (6)	30	4	4/39	4	$8.95

final tree and the grader reads what the gate read, and the selection reveals roughly which files the hidden test patch touches; a gate on the full suite, or on tests chosen without benchmark metadata [8], can only do worse. A fifth arm, a split variant, removes the guarantee: the gate reads a deterministic half of the previously-passing ids (2,278) and never the other half (2,585), on which the grader also rules [45]. It resolved **28 of 40 (70.0%)**, indistinguishable from the full gate (5 discordant pairs to 3, $p = 0.73$) and better than plain rollback (13 to 2, McNemar $p = 0.007$); no final tree failed a held-out test apart from the parser-capped instance of Section 5.1.

5.7 Replication on SWE-bench Live

Live keeps the undercount and loses both the resolve rates and the stack difference (Table 1). Resolve collapsed for every arm, to at most 6 of 40 even under the rot-aware count against 13 to 26 for the same four arms on Verified, so the recovery contrast on resolve is uninformative on Live; the collapse is consistent with training-data contamination of Verified [6, 46], though Live's larger oracles and 15 baseline-dead cells mean the substrates differ in more than contamination. The undercount replicated: GPT-5.6 rollback recorded 156 events in 9 incidents across 6 of 39 runs and left no contaminated final tree, while no recovery left one in each of its 4 bearing runs (worse than rollback on 4 instances, better on 0; sign test $p = 0.125$). The gated arm, clean by construction on Verified, left two contaminated trees on Live against rollback's none (sign test $p = 0.50$) and resolved 1 (4) against 0 (2): one run was cut off by the $4 cap before the gate saw its final tree; the other passed every gate check yet fails 13 tests under the grader, 7 in the file the hidden test patch rewrites. The stack difference did not: the Claude stack, silent on Verified, was bearing on 3 of the 39 shared Live instances (49 events) against GPT-5.6's 6 of 39 (one-sided Fisher $p = 0.24$). The Verified zero and Live's 3 of 39 are not distinguishable (the exact interval for 0 of 40 reaches 8.8%): memorised fixes on Verified and a cleaner stack on both substrates both fit.

6 Discussion

Why timeline regressions matter. A repaired regression is not free: 27% of spend across the eight Verified bearing runs ($0.63 of $2.33) went to work done while a regression was open. The events are the per-step ground truth patch verifiers [24] and process reward models [25] need; the timeline is what tells an agent-OS recovery primitive such as a branch context [42] when to abort. Two rules would have prevented most of the 28 defects met while building the instrument (Appendix A): record infrastructure failures as missing observations, and require positive evidence that the measurement path is live.

Limitations. Each substrate has a 40-instance slice and one sweep per arm; the cross-stack comparisons rest on six and nine bearing runs, confounded with the provider adapter; the gated arms are post hoc and select tests from benchmark metadata; Live resolve is near floor; runs are ten times shorter than public-scaffold trajectories [32–34]; intervals ignore repository clustering; event counts are lower bounds; a public-scaffold run [9, 10] is next.

References

[1] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR 2024. arXiv:2310.06770.

[2] OpenAI. Introducing SWE-bench Verified. OpenAI blog, August 2024. openai.com/index/introducing-swe-bench-verified.

[3] L. Zhang et al. SWE-bench Goes Live! NeurIPS 2025 Datasets and Benchmarks. arXiv:2505.23419.

[4] X. Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? arXiv:2509.16941, 2025.

[5] I. Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. NeurIPS 2025. arXiv:2505.20411.

[6] S. Liang et al. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of reason. arXiv:2506.12286, 2025.

[7] B. Yu et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. ACL 2025. arXiv:2506.09289.

[8] P. Alonso, S. Yovine, and V. A. Braberman. TDAD: Test-driven agentic development: Reducing code regressions in AI coding agents via graph-based impact analysis. arXiv:2603.17973, 2026.

[9] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024. arXiv:2405.15793.

[10] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. ICLR 2025. arXiv:2407.16741.

[11] C. S. Xia et al. Agentless: Demystifying LLM-based software engineering agents. FSE 2025. arXiv:2407.01489.

[12] K. Lieret and C. E. Jimenez. mini-swe-agent. Software, 2025. github.com/SWE-agent/mini-swe-agent.

[13] X. Wang et al. Executable code actions elicit better LLM agents. ICML 2024. arXiv:2402.01030.

[14] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. ICLR 2023. arXiv:2210.03629.

[15] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023. arXiv:2303.11366.

[16] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023. arXiv:2303.17651.

[17] A. Kadu and A. Krishnan. ReflexGrad: Within-episode failure recovery in LLM agents via progress-gated dual-process routing. arXiv:2511.14584, 2025.

[18] P. Mazaheri. REPOT: Recoverable program-of-thought via checkpoint repair. arXiv:2605.30052, 2026.

[19] Y. Wang et al. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. arXiv:2606.04990, 2026.

[20] K. Mei et al. AIOS: LLM agent operating system. COLM 2025. arXiv:2403.16971.

[21] S. Kapoor et al. Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. arXiv:2510.11977, 2025.

[22] R. Shu et al. What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv:2607.06184, 2026.

[23] T. Le et al. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470, 2025.

[24] M. Raghavendra et al. Agentic rubrics as contextual verifiers for SWE agents. ACL 2026. arXiv:2601.04171.

[25] M. L. Dihan and M. A. R. Khan. SWE-Shepherd: Advancing PRMs for reinforcing code agents. arXiv:2604.10493, 2026.

[26] Z. Qi, F. Long, S. Achour, and M. Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. ISSTA 2015. doi:10.1145/2771783.2771791.

[27] Y. Lou et al. When automated program repair meets regression testing: An extensive study on two million patches. ACM TOSEM 33(7), 2024. arXiv:2105.07311.

[28] Z. Fei et al. Patch correctness assessment: A survey. ACM TOSEM 34(2), 2025. doi:10.1145/3702972.

[29] H. Ye, M. Martinez, and M. Monperrus. Automated patch assessment for program repair at scale. Empirical Software Engineering 26(2), 2021. doi:10.1007/s10664-020-09920-w.

[30] SWE-bench issue #601: Test result hijacking via stdout forging in evaluation harness. github.com/SWE-bench/SWE-bench/issues/601, June 2026.

[31] OpenHands issues #4235 (October 2024) and #7044 (March 2025): Docker and environment errors in SWE-bench instance evaluation. github.com/OpenHands/OpenHands.

[32] J. Yang et al. SWE-smith: Scaling data for software engineering agents. NeurIPS 2025 Datasets and Benchmarks. arXiv:2504.21798.

[33] J. Pan et al. Training software engineering agents and verifiers with SWE-Gym. ICML 2025. arXiv:2412.21139.

[34] S. Liu et al. Context as a tool: Context management for long-horizon SWE-agents. Findings of ACL 2026. arXiv:2512.22087.

[35] M. B. Madiraju and M. S. P. Madiraju. RigorBench: Benchmarking engineering process discipline in autonomous AI coding agents. arXiv:2606.22678, 2026.

[36] J. Chen et al. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration. arXiv:2603.03823, 2026.

[37] Y. Wang, M. Pradel, and Z. Liu. Are "solved issues" in SWE-bench really solved correctly? An empirical study. ICSE 2026. arXiv:2503.15223.

[38] P. Sahoo et al. AgentLens: Revealing the lucky pass problem in SWE-agent evaluation. arXiv:2605.12925, 2026.

[39] M. Kim et al. Coherence collapse: Diagnosing why code agents fail after reaching the right code. arXiv:2603.24631, 2026.

[40] X. Gao, J. Yang, and Q. Yang. Looping is not reliability: State-bound evidence and typed revision contracts for agentic code repair. arXiv:2607.24604, 2026.

[41] Y. Zhuang et al. AgentRewind: Recoverable execution for long-horizon LLM agents. arXiv:2608.14380, 2026.

[42] C. Wang and Y. Zheng. Fork, explore, commit: OS primitives for agentic exploration. arXiv:2602.08199, 2026.

[43] Y. Chen, T. Ahmed, R. Jabbarvand, and M. Hirzel. Can old tests do new tricks for resolving SWE issues? FSE 2026. arXiv:2510.18270.

[44] P. Gao et al. Trae Agent: An LLM-based agent for software engineering with test-time scaling. arXiv:2507.23370, 2025.

[45] T. Ahmed, J. Ganhotra, A. Shinnar, and M. Hirzel. Investigating test overfitting on SWE-bench. arXiv:2511.16858, 2025.

[46] OpenAI. Why SWE-bench Verified no longer measures frontier coding capabilities. OpenAI blog, February 2026. openai.com/index/why-we-no-longer-evaluate-swe-bench-verified.

A The measurement defect catalogue

A.1 The catalogue by mechanism

Each row is a defect found while building the instrument. **S**: it produced a plausible number rather than an error. **G**: it was present while the test suite passed (A4 is the suite). **H**: it was findable only on a clean host. Class A rows depend on the machine the code runs on; class B rows render an infrastructure failure as a measurement; class C rows measure something other than the event as defined; class D rows consume a producer's output without checking the producer.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherited the machine's git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Validation probe invoked bare `python`	every probe exit 127 on a clean host	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare `pytest` from PATH	15 tests fail on a clean host as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare `python`	score 0.0 from a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the container	26 of 28 zero-step runs; see A.2	✓	✓	✓
B1	Output marker used shell `:`	a 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework's results outside the parsed slice	✓	✓	
B3	Probe diffed against a deleted commit	every graded test a hole	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances fail far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127	✓	✓	
B6	Isolation removed loopback, not only the internet	23.5% of the oracle dead	✓	✓	
B7	Provider cache served removed containers	later cells replay against nothing and score clean	✓	✓	
B8	Tree reset reverted image build-time edits	one repository family's oracle dead	✓	✓	
C1	Bisection assumed monotone verdicts	recovered regressions invisible	✓	✓	
C2	Declared event unit not implemented	rate off by up to 2.7×	✓	✓	
C3	Rollbacks produced no observation	recoveries invisible to the timeline	✓	✓	
C4	"Persists past the step boundary" undefined	one reading excludes the events of interest	✓	✓	
D1	Audit field never populated	0 tool errors reported, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost as task failures	✗	✓	
D3	Dependency install counted as agent work	attribution vacuous	✓	✓	
D4	Join took the first observation, not the last	failures pinned to the wrong tree	✓	✓	
D5	Executed config rebuilt from a name	manifest described a different run	✓	✓	
D6	Git handles never released	sweeps die after ~400 cells	✓	✓	
D7	Console re-walked the tree per run	presented as a dead server	✗	✓	
D8	Detection events lacked the ids attribution joined on	attributed detection always unknown	✓	✓	
D9	Absent coverage rendered as measured silence	unmeasured regressions reported as silent	✓	✓	
D10	Output-capped turns executed	half-written file; a 78-event storm	✓	✓	

Totals: 23 of 28 silent; 27 of 28 present under a passing suite; 6 of 28 findable only on a clean host. Eight further defects found after this census closed are described where they arose (Sections 5.1 and 5.4 and Appendix A.3); the rest will be released with the code.

A.2 The defect that invalidated a conclusion

After nineteen fixes, every validation check passed, the re-scoring control reproduced archived episode counts exactly, and three pilots (80 runs) measured an event rate far below the pre-declared proceed criterion. We drafted the conclusion the rule directed: the benchmark offered too little opportunity, so switch benchmarks. The conclusion was wrong. The harness ran the agent on the host in a bare source checkout, while every probe and every gate ran inside the pinned container. On the host, `import matplotlib` in that checkout succeeds by importing the uncompiled source tree as a namespace package, and everything downstream fails in ways indistinguishable from an incompetent

agent. Twenty-six of 28 zero-step runs traced to this. The checks had validated the measurement path and never the agent's execution path. The fix was to route the agent's tools and checks through the same container as the replay, and to add a per-cell environment parity check that runs before any model call.

A.3 Mechanisms in public infrastructure

Class A: OpenHands issues #4235 and #7044 [31], environment construction failures against SWE-bench images. Class B: UTBoost's finding that the held-out oracle is insufficient at leaderboard scale [7], and, on SWE-bench Live, baseline tests that fail in the unmodified image because the calendar passed a deprecation date encoded in the instance. Class C: final-state regression measurement [8]. Class D: the SWE-bench grader accepting forged test output on stdout [30], and, at the harness's current revision, a parser that keys parametrised ids by their full text while the dataset stores them truncated at the first space (the earlier parser's behaviour), so 16 previously-passing ids of one instance in our slice match no output and every submission to it grades as unresolved.

B Pre-declaration and analysis

The event unit is one (test function, onset observation) pair with parametrised variants collapsed. The gates for proceeding on a substrate were an event rate of at least 0.30 per run and a bearing fraction of at least 25%. Both stacks failed the conjunction (bearing 0% and 16.2%); the gates were then re-declared per (substrate, model stack) before the contrast was unblinded, and no arm reported here is confirmatory. The contrast's primary endpoint is final-state contamination, paired by instance, exact two-sided sign test; the co-primary is incident exposure. The analysis script was committed before either comparison arm finished. The regression-gated arm was added after the contrast was unblinded and is exploratory. No multiplicity adjustment was planned or applied. Budget exhaustion is scored as failure. Instances whose baseline oracle records failures in the unmodified image are excluded from resolve-rate comparability claims, and their dead tests are typed as missing observations for event counting.

Compute. Every run, replay, and grade executed on one 12-vCPU, 22 GB x86 virtual machine running Docker, with no GPUs; the six Verified arms in Table 1 cost $81.42 in model spend plus $6.87 for the second rollback sweep, the calibration and pilots a comparable amount, and the Live sweeps are reported with their own spend in Table 1.

Grading correction between unblindings. The first unblinding of the recovery-policy contrast gave $p = 0.375$ for the primary endpoint: seven cells (five no-recovery, two repair-in-place) had been dropped as ungradable because their final trees made the suite uncollectable. Official grading scores such a patch as failing every test, so the most contaminated states had been dropped from the endpoint they bore on most. The grader, not the replay, now distinguishes an environment failure from a patch that kills the suite, and the seven archived trees were re-graded without re-running any agent.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state the scope (a pilot on a 40-instance slice per substrate, one sweep per arm), the four contributions, and that all measurements are exploratory; Section 6 lists the limitations that bound them.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 covers slice size, single sweeps, the adapter confound of the cross-stack comparison, the post hoc gated arms and their selection leakage, trajectory length relative to public scaffolds, Live resolve near floor, and event counts as lower bounds; Section 5.4 discloses the grading correction made after unblinding.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 3 specifies the instrument (commit per mutating call, replay of previously-passing tests in the pinned container, attribution, validation gates), Section 4 the substrates, slice rules, harness, tools, model identifiers, cost caps, outcomes and pre-declaration, and Appendix B the analysis plan; the defect catalogue (Appendix A) records the measurement pitfalls a replication must avoid.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: The benchmark data are public (SWE-bench Verified, SWE-bench Live). The instrument, the sweep driver, the analysis scripts and the archived timelines will be released with the camera-ready version; they are not attached to the anonymous submission.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: No models are trained. Section 4 gives the slices and how they were fixed, the harness and its three recovery policies (the two regression-gated variants are specified in Section 5.6), the exact model strings, the per-run cost cap, the outcome definitions, and the pre-declared analysis.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: Resolve and bearing fractions carry exact (Clopper–Pearson) 95% intervals; paired comparisons use exact sign tests and exact McNemar tests, the cross-stack comparison a one-sided Fisher exact test with its sensitivity to reassigned runs stated; the tests reported and the absence of multiplicity adjustment are declared (Section 5.6, Appendix B). Run-to-run variability is measured only for plain rollback (Section 5.3), and intervals treat instances as independent although 16 of the 40 Verified instances come from django (Section 6).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 3 gives replay cost per observation and per sweep; Table 1 gives model spend per arm and substrate; Appendix B states the machine (one 12-vCPU, 22 GB x86 virtual machine running Docker, no GPUs) and the total model spend including pilots.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The work measures software agents on public benchmarks; no human subjects, personal data, or high-risk assets are involved, and the submission is anonymized.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [No]

    Justification: The contribution is an evaluation instrument and measurements of it. The plausible impact is positive (agents that break less existing behaviour); we see no direct path to a negative application, and the paper does not discuss societal impact.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: No models or scraped datasets are released.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: SWE-bench, SWE-bench Verified and SWE-bench Live are cited [1, 2, 3] and used under their MIT licenses and published container images; the models are accessed through their providers' APIs under the providers' terms; every cited artefact appears in the references.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [No]

Justification: The instrument and the archived timelines are not released with the anonymous submission; they will be released, documented, with the camera-ready version.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing or human-subject research.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: LLMs are the agents under measurement: the planner, worker and monitor models are named by their API identifiers in Section 4, and their roles in the harness are described in Sections 3 and 4.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.